```
object CargosPerContainerProjection {

  def createProjectionFor(                                    ❶
      system: ActorSystem[_],
      repository: CargosPerContainerRepository,
      indexTag: Int): ExactlyOnceProjection[
    Offset,
    EventEnvelope[SPContainer.Event]] = {

    val tag = "container-tag-" + indexTag                     ❷

    val sourceProvider =                                      ❸
      EventSourcedProvider.eventsByTag[SPContainer.Event](
        system = system,
        readJournalPluginId = JdbcReadJournal.Identifier,
        tag = tag)

    JdbcProjection.exactlyOnce(                               ❹
      projectionId =
        ProjectionId("CargosPerContainerProjection", tag),
      sourceProvider = sourceProvider,
      handler =
        () => new CPCProjectionHandler(repository),
      sessionFactory = () => new ScalikeJdbcSession())(system)
  }
}
```

❶ Instantiates the projection

❷ Filter event definition

❸ Reads from the journal

❹ Writes to the database

Depending on the index you pass to this method, a projection is created for different events. If you specify index `1`, a projection is created that reads only events with the tag `container-tag-1`. An interesting question is how to create multiple projections, one per tag, running on different nodes to scale out and isolate possible failures.

To do this, you can reuse an idea you know about: sharding—spawning behaviors evenly in a cluster. Instead of Akka Sharding, you can use `ShardedDaemonProcess`, which is similar to Akka Sharding on the spawning side but significantly different in terms of communication.

As far as spawning is concerned, they are similar in that they distribute the entities. However, `ShardedDaemonProcess` has limitations. It needs to know in advance, when it is created, how many instances of the behavior you want to create. This is not the case with Akka Sharding; you can create as many sharded actors as you need.

On the communication side, Akka Sharding has the sharded region (SR), which you use as a proxy to send messages to actors. However, the `ShardedDaemonProcess` does not provide this capability. You cannot send messages directly to these behaviors; they are processes implemented in Akka Streams. With a `ShardedDaemonProcess`, you use the receptionist you saw in chapter 6, but that's secondary now.

10.3.4 The ShardedDaemonProcess

To understand the `ShardedDaemonProcess` more quickly without the added complexity of the `SPContainer`, let's look at a minimal example with only one behavior spawned by the